

Seguridad en APIs: pon los límites

Proyecto: MediReserva — Autorización y cierre

Segundo momento del taller

La semana pasada construiste la fundación y la autenticación de MediReserva: los usuarios se registran e inician sesión con JWT. Hoy completas el sistema — que cada quien haga SOLO lo suyo.

Sigues sin recibir código en este documento. Tienes el nuevo encargo del cliente, tu misión, los comandos para crear los archivos que faltan, pistas que guían (no resuelven) y el checklist para saber si lo lograste.

Continúas el MISMO proyecto medireserva_api. No creas nada desde cero: hoy proteges y completas lo que ya tienes. Apóyate en devacademy_api (Sesión 2) como tu mejor referencia.

0. Retoma — ¿Dónde quedaste?

Antes de avanzar, reubícate. La semana pasada dejaste MediReserva así:

- Las entidades modeladas (Usuario con rol, y las que conectan pacientes con médicos y horarios).
- La capa de seguridad lista: hashéo de contraseñas y creación de tokens.
- Registro y login funcionando: un usuario se registra e inicia sesión, y recibe su token.

Pero hay algo abierto: en este momento, cualquiera con un token puede hacer cualquier cosa. No hay endpoints protegidos. Un paciente podría ver las citas de otro. Un médico podría tocar la agenda ajena. Eso es justo lo que el cliente NO quiere. Hoy lo resuelves.

Las tres piezas que hoy cobran sentido

Recuerda que en la Parte 1 pusiste, a propósito, cosas que 'todavía no usabas'. Hoy son las protagonistas:

1. El campo 'rol' en Usuario → hoy decide qué puede hacer cada quien.
2. Las llaves foráneas de dueño en tus entidades (el paciente de una cita, el médico de un horario) → hoy deciden quién toca qué.
3. El 'sub' del token (el id del usuario) → hoy es la fuente de identidad de cada petición.

Toda la autorización de hoy sale de cruzar el 'sub' del token con esas llaves foráneas. Igual que en DevAcademy.

1. Un nuevo correo del cliente

El cliente vio tu avance de la semana pasada y quedó contento. Ahora te escribe con la parte que más le importa.

Nuevo mensaje — Cliente: Clínica MediReserva

De: Administración de Clínica MediReserva

Para: Equipo de desarrollo

Asunto: Ahora sí, la seguridad de verdad

Hola de nuevo,

Vimos que ya se puede iniciar sesión. Excelente. Pero necesitamos lo más importante para una clínica: la privacidad tiene que funcionar de verdad.

Recuerden las reglas: un médico crea y gestiona SU agenda de horarios, y nadie más puede tocarla. Un paciente agenda citas y solo puede ver y cancelar LAS SUYAS, jamás las de otro paciente. El administrador supervisa a los usuarios del sistema.

También nos preocupa algo práctico: si un médico está a mitad de su jornada y su sesión se vence, no queremos que tenga que volver a escribir su clave a cada rato. Debe haber una forma de mantener la sesión de forma segura.

Y por favor, que cuando alguien intente hacer algo que no le corresponde, el sistema responda de forma clara y profesional, no con un error cualquiera.

Gracias por el buen trabajo. Confiamos en ustedes.

— Administración, Clínica MediReserva

2. Antes de programar — Traduce el nuevo encargo

Como siempre, primero traduces lo que el cliente pide a decisiones técnicas. Este correo esconde cuatro necesidades.

Tu ejercicio de análisis (antes de tocar código)

Empareja cada frase del cliente con el concepto técnico que ya conoces:

- 'un médico gestiona SU agenda, nadie más' → ¿qué tipo de autorización es? ¿qué comparas?
- 'un paciente solo ve LAS SUYAS' → ¿de dónde sale el id del paciente para filtrar: del cliente o del token?
- 'que no tenga que escribir su clave a cada rato' → ¿qué mecanismo de tokens resuelve esto?
- 'que responda de forma clara y profesional' → ¿qué código HTTP y qué encabezado usas cuando alguien no tiene permiso?

Las cuatro las resolviste en DevAcademy. Hoy las aplicas en la clínica. Ese es el punto: demostrarte que el conocimiento se traslada.

3. Tu misión de HOY (Parte 2)

Hoy proteges MediReserva completa. Al terminar, cada rol hace exactamente lo que le corresponde y nada más.

Lo que debes lograr hoy

- Crear la dependencia que saca al usuario desde el token (la que reconoce quién hace la petición).
- Crear las dependencias de rol (una para médico, una para admin).
- Agregar el endpoint de refresh token (las piezas ya las tienes de la Parte 1).
- Proteger los horarios: un médico crea los suyos y solo edita/elimina los suyos.
- Proteger las citas: un paciente agenda, y solo ve y cancela las suyas.
- Estandarizar los errores de autenticación (401 con el encabezado correcto, 403 cuando no corresponde).
- Conectar los routers nuevos y probar los tres roles chocando contra los permisos.

4. Comandos — Crear los archivos que faltan

Ya tienes el proyecto montado desde la Parte 1. Hoy solo agregas los archivos nuevos. Recuerda la regla del taller: el comando crea el archivo vacío; el código lo escribes tú.

Activa tu entorno y ubícate en el proyecto

```
cd medireserva_api
venv\Scripts\activate
```

Creas los archivos nuevos (adáptalos a tus nombres de entidades)

Estos son los archivos que probablemente te faltan. Si nombraste tus entidades distinto, ajusta los nombres.

```
# La dependencia de seguridad (este archivo ya existe pero está vacío)
# Ábrelo y complétalo: app\auth\dependencias.py

# Services y routers de tus entidades protegidas
# (ejemplo con horarios y citas; usa TUS nombres):
New-Item app\services\horario_service.py, app\routers\horario_router.py -ItemType File
New-Item app\services\cita_service.py, app\routers\cita_router.py -ItemType File
```

No olvides

Al final, tienes que registrar los routers nuevos en main.py con `include_router`, o los endpoints no aparecerán en Swagger. Es el error más común al cerrar el proyecto.

5. ¿Cómo sabes que lo lograste? — Checklist de hoy

Marca cada punto. Si todos quedan marcados, MediReserva está completo y seguro.

Reconocimiento y roles

- ☐ Existe una dependencia que, dado un token válido, devuelve el usuario; y con token inválido responde 401.
- ☐ Hay una dependencia que exige rol de médico y otra que exige rol de admin.

- ☐ Un endpoint protegido, sin token, responde 401 con el encabezado WWW-Authenticate.

Autorización de médicos (su agenda)

- ☐ Un médico puede crear un horario, y ese horario queda a su nombre (el id sale del token, no del body).
- ☐ Un médico NO puede editar ni eliminar un horario de otro médico (responde 403).
- ☐ Un paciente NO puede crear horarios (responde 403 por rol).

Autorización de pacientes (sus citas)

- ☐ Un paciente puede agendar una cita, y queda a su nombre (id desde el token).
- ☐ Un paciente solo ve SUS citas; no hay forma de ver las de otro paciente.
- ☐ Un paciente NO puede cancelar la cita de otro paciente (responde 403).

Sesión y errores

- ☐ Existe el endpoint de refresh: con un refresh token válido devuelve un access token nuevo.
- ☐ Enviar un access token al endpoint de refresh es rechazado (cada token en su lugar).
- ☐ Los intentos sin permiso devuelven 403; los de identidad fallida devuelven 401. No se confunden.

Pruebas de los tres roles

- ☐ Probaste en Postman el recorrido completo: médico, paciente y admin contra los permisos.
- ☐ Confirmaste en Swagger con el botón Authorize.
- ☐ Viste al menos un caso donde el MISMO endpoint da 200 o 403 según de quién sea el dato.

6. Pistas — Si te atascas (dirección, no código)

Intenta primero por tu cuenta. Tu mejor referencia es la Sesión 2 de DevAcademy: allí hiciste exactamente esto con cursos, lecciones e inscripciones.

Pista 1 — La dependencia que reconoce al usuario

El portero interno

Necesitas una función que reciba el token, lo verifique, saque el 'sub' (id), busque al usuario en la base y lo devuelva. Si algo falla, 401.

Es la misma pieza que en DevAcademy llamaste 'obtener_usuario_actual'. Cópiala mentalmente de ahí y adáptala. Sobre ella construyes las dependencias de rol: una que exija médico, otra que exija admin.

Pista 2 — El permiso de la agenda del médico

Autorización a nivel de dato

Cuando un médico quiera editar o eliminar un horario, no basta con que sea médico. Tienes que verificar que ESE horario sea suyo.

¿Cómo? Compara el campo de dueño del horario (el id del médico que lo creó) con el id del usuario del token. Si no coinciden, 403. Es la misma comparación que hiciste con instructor_id en los cursos de

Pista 3 — El paciente solo ve lo suyo

El filtro sale del token

Para listar las citas de un paciente, filtra la consulta por el id del paciente que viene del token, no por un id que mande el cliente. Así es imposible pedir 'las citas del paciente 7'.

Idéntico a 'mis-inscripciones' en DevAcademy y a 'mis-préstamos' en biblioteca. Reconoce el patrón: cada quien lo suyo, con el id del token.

Pista 4 — Mantener la sesión sin reloguear

El refresh token

El cliente pidió que el médico no tenga que escribir su clave a cada rato. Eso es el refresh token: un endpoint que recibe el refresh token y devuelve un access token nuevo.

Las funciones para crear y verificar tokens ya las tienes de la Parte 1 (incluyendo la distinción de tipo access/refresh). Solo te falta el endpoint que las use. Míralo en el auth_router de DevAcademy Sesión 2.

Pista 5 — Errores profesionales

401 vs 403, y el encabezado

401 = 'no sé quién eres' (token ausente, inválido o expirado). 403 = 'sé quién eres, pero esto no te corresponde' (rol o dato ajeno). No los confundas: son cosas distintas.

Cuando devuelvas un 401 en tus dependencias, incluye el encabezado WWW-Authenticate con valor Bearer. Es el estándar que hace que tu 401 sea profesional y no improvisado.

Pista 6 — El orden de construcción sugerido

- Completa la dependencia de seguridad (reconocer usuario + dependencias de rol).
- Agrega el endpoint de refresh al router de autenticación.
- Protege los horarios: service con la verificación de dueño, router con la dependencia de médico.
- Protege las citas: service que filtra por el paciente del token, router con la dependencia de usuario.
- Registra los routers nuevos en main.py.
- Levanta, prueba los tres roles, celebra: terminaste tu tercer proyecto de seguridad.

7. Reto opcional — Para los que quieran ir más allá

Si terminas con tiempo y quieres un desafío real, intenta esto (es autorización anidada, como las lecciones de DevAcademy):

Desafío avanzado

Haz que un paciente solo pueda agendar una cita en un horario que de verdad esté disponible (que exista

y no esté ya tomado). Y que, al agendar, ese horario quede marcado como ocupado.

Pista: antes de crear la cita, tu service tendría que buscar el horario, verificar su estado, y solo entonces crear la cita y actualizar el horario. Es lógica de negocio + seguridad trabajando juntas, como el descuento de stock que hiciste en biblioteca.

8. Cierre — Lo que lograste

Si completaste el checklist, MediReserva ya no solo sabe quién es cada quien: ahora pone límites. El médico gestiona su agenda, el paciente sus citas, el admin supervisa. Exactamente lo que el cliente pidió.

Y lo más importante: este es tu tercer proyecto de seguridad. Lo construiste tú, casi sin ayuda, en un negocio distinto a los anteriores. Eso demuestra que no memorizaste biblioteca ni DevAcademy: aprendiste a pensar la seguridad. Esa es la habilidad que te llevas, y la que te hace un desarrollador backend de verdad.

Tres proyectos, un mismo criterio. Ya estás listo para el proyecto final del programa.
